

876. Middle of the Linked List Easy

Acceptance: 81.4% | [LeetCode](#)

Given the `head` of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:

```
**Input:** head = [1,2,3,4,5]
**Output:** [3,4,5]
**Explanation:** The middle node of the list is node 3.
```

Example 2:

```
**Input:** head = [1,2,3,4,5,6]
**Output:** [4,5,6]
**Explanation:** Since the list has two middle nodes with values 3 and 4, we return the second one.
```

Constraints:

- The number of nodes in the list is in the range `[1, 100]`.
- `1 <= Node.val <= 100`

Claude Opus Solution

Middle of the Linked List

1. Intuition

The core challenge: we can't index into a linked list like an array, and we don't know the length upfront without traversing it first.

Key Insight — The "Tortoise and Hare" (Fast & Slow Pointer) pattern:

If one pointer moves **twice as fast** as another, then when the fast pointer reaches the **end**, the slow pointer will be exactly at the **middle**.

Think of it like two runners on a track: by the time the fast runner finishes the full distance, the slow runner (at half speed) has covered exactly half the distance — landing precisely in the middle.

This elegantly handles the "return the second middle node" requirement for even-length lists, as we'll see in the complexity analysis.

2. Approach (Pseudocode)

- Initialize two pointers, slow and fast, both at head.
- Loop while fast and fast.next are both valid:
 - Move slow forward by 1 node.
 - Move fast forward by 2 nodes.
- When the loop ends, slow points to the middle node.
- Return slow.

Why check both fast and fast.next? - `fast` handles **even-length** lists (fast lands exactly on `None`). - `fast.next` handles **odd-length** lists (fast lands on the last node, where `fast.next` is `None`).

3. Python Solution

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # Both pointers start at the head of the list.
        slow = fast = head

        # Continue as long as fast can take TWO more steps:
```

```

# - 'fast' must exist (so fast.next is valid to access)
# - 'fast.next' must exist (so fast.next.next is valid)
while fast and fast.next:
    slow = slow.next          # slow moves 1 step
    fast = fast.next.next     # fast moves 2 steps

# When the loop exits:
# - Odd length: fast is on the last node, slow is at the true middle.
# - Even length: fast is None, slow is at the SECOND of two middles.
return slow

```

Step-by-Step Trace

Odd: [1,2,3,4,5] | Step | slow | fast | |-----|-----|-----| | Start | 1 | 1 | | Iter 1 | 2 | 3 | | Iter 2 | 3 | 5 | | Stop (fast.next is None) | 3 ✓ | 5 |

Even: [1,2,3,4,5,6] | Step | slow | fast | |-----|-----|-----| | Start | 1 | 1 | | Iter 1 | 2 | 3 | | Iter 2 | 3 | 5 | | Iter 3 | 4 | None | | Stop (fast is None) | 4 ✓ | None |

Notice slow naturally lands on the **second** middle (4) for even lists — exactly what the problem requires.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	The fast pointer traverses the list at 2 steps per iteration, so we loop $\sim n/2$ times $\rightarrow$ linear in the number of nodes.
Space	$O(1)$	We only use two pointer variables, regardless of list size. No extra data structures.

5. Alternative Approaches

Approach A: Two-Pass (Count then Walk)

```

def middleNode(self, head):
    # Pass 1: count the nodes
    n = 0
    node = head
    while node:
        n += 1
        node = node.next

    # Pass 2: walk to index n // 2 (this gives the second middle for even lengths)
    node = head
    for _ in range(n // 2):
        node = node.next
    return node

```

- **Time:** $O(n)$ (two passes), **Space:** $O(1)$
- Simpler to reason about, but requires traversing the list twice.

Approach B: Store Nodes in an Array

```

def middleNode(self, head):
    nodes = []
    while head:
        nodes.append(head)
        head = head.next
    return nodes[len(nodes) // 2]

```

- **Time:** $O(n)$, **Space:** $O(n)$ — wastes memory storing all nodes.

Why the Fast/Slow Pointer Wins

It achieves the answer in a **single pass** with **constant space**, and naturally handles the even-length edge case. This same pattern is foundational for many linked-list problems: - **Cycle detection** (LeetCode 141) - **Finding cycle start** (LeetCode 142) - **Nth node from the end** (LeetCode 19) - **Palindrome linked list** (LeetCode 234)

Mastering this pattern unlocks an entire category of problems.

Codex Solution

1. Intuition

The key pattern is **Fast and Slow Pointers**.

We use two pointers:

- `slow` moves **1 node at a time**
- `fast` moves **2 nodes at a time**

Because `fast` moves twice as quickly, by the time `fast` reaches the end of the list, `slow` will be at the middle.

For an even-length list, this naturally returns the **second middle node**.

Example:

```
head = [1, 2, 3, 4, 5, 6]
```

Start:

```
slow = 1  
fast = 1
```

Step 1:

```
slow = 2  
fast = 3
```

Step 2:

```
slow = 3  
fast = 5
```

Step 3:

```
slow = 4  
fast = None
```

```
Return slow = 4
```

So the returned list starts from node 4: [4, 5, 6].

2. Approach

Pseudocode

```
function middleNode(head):  
    slow = head  
    fast = head  
  
    while fast is not null and fast.next is not null:  
        slow = slow.next  
        fast = fast.next.next  
  
    return slow
```

Why this works

- `fast` moves two steps for every one step of `slow`.
- When `fast` reaches the end, `slow` has moved half as many steps.
- Therefore, `slow` points to the middle node.
- If there are two middle nodes, `slow` lands on the second one.

3. Python Solution

```
from typing import Optional
```

```
# Definition for singly-linked list.
```

```
class ListNode:
```

```
    def __init__(self, val: int = 0, next: Optional["ListNode"] = None):  
        self.val = val
```

```
self.next = next
```

```
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # Both pointers start at the head of the linked list.
        slow = head
        fast = head

        # Move fast by 2 nodes and slow by 1 node.
        #
        # We continue while fast can still move two steps:
        # - fast must not be None
        # - fast.next must not be None
        #
        # When fast reaches the end, slow will be at the middle.
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # slow now points to the middle node.
        # If the list has even length, this will be the second middle node.
        return slow
```

4. Complexity Analysis

Time Complexity: $O(n)$

We traverse the linked list once.

Although `fast` moves two nodes at a time, the total number of operations is still proportional to the number of nodes `n`.

So the time complexity is:

```
 $O(n)$ 
```

Space Complexity: $O(1)$

We only use two pointers:

- `slow`
- `fast`

No extra data structures are used.

So the space complexity is:

```
 $O(1)$ 
```

5. Alternative Approaches

Approach 1: Count the Length First

You can first traverse the list to count the number of nodes.

Then move `length // 2` steps from the head.

```
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        length = 0
        current = head

        # First pass: count nodes
        while current:
            length += 1
            current = current.next

        # Move to the middle
        middle_index = length // 2
        current = head

        for _ in range(middle_index):
            current = current.next
```

```
return current
```

Complexity:

```
Time: O(n)  
Space: O(1)
```

Downside: requires two passes through the list.

Approach 2: Store Nodes in an Array

Store every node in a list, then return the node at index `n // 2`.

```
class Solution:  
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:  
        nodes = []  
        current = head  
  
        while current:  
            nodes.append(current)  
            current = current.next  
  
        return nodes[len(nodes) // 2]
```

Complexity:

```
Time: O(n)  
Space: O(n)
```

Downside: uses extra memory.

Key Pattern

This problem is a classic example of the:

```
Fast and Slow Pointers pattern
```

This pattern is commonly used for linked list problems such as:

- Finding the middle of a linked list
- Detecting cycles in a linked list
- Finding the start of a cycle
- Checking linked list palindromes in optimized solutions